Build an academic workspace and semester planner across Google Drive, Word, Excel, and Slides with automated assignment tracking

Computer
Build a course marketplace and research studio platform with integrated Google Drive sync, assignment generators, and rubric scoring
Computer
Show me the architecture plan for the editor modules
Draft the core data model for assignments and documents
Detail how the policy-controlled AI layer works

Build this as one native academic **workspace** with four connected systems:

1. Editor modules for documents, spreadsheets, presentations, and research artifacts.
2. A semester planner that turns verified assignment information into tasks, milestones, and calendar plans.
3. A course marketplace and Research Studio that reuse the same sources, artifacts, permissions, and learning objects.
4. A policy-controlled AI gateway that governs generation, feedback, retrieval, and actions.

Google Drive should be an optional, explicitly authorized synchronization bridge. Word, Excel, and PowerPoint should be supported import/export formats—not the underlying authority for every Semester object.

This is an implementation specification, not a claim of executed code or deployment. The project's October 4 architecture snapshot describes an existing modular-monolith direction, but also reports incomplete platform adoption, an unbuilt marketplace, and unresolved tenant AI-policy enforcement. Those dated findings need revalidation before activation.[1]

1. Overall architecture

```
Semester application
│
├── Academic workspace
│   ├── Files and collections
│   ├── Docs editor
│   ├── Sheets editor
│   ├── Slides editor
│   ├── Research Studio
│   └── Assignment workspace
```

```
│
├── Semester planner
│   ├── Assignment intake and verification
│   ├── Milestones and tasks
│   ├── Calendar and workload planning
│   └── Action Center
│
├── Course ecosystem
│   ├── Course Studio
│   ├── Course marketplace
│   ├── Learning resources
│   └── Rubrics and assessment workflows
│
└── Shared application services
    ├── Identity, ownership, membership, consent
    ├── Artifact versions and collaboration
    ├── Search and source provenance
    ├── Policy-controlled AI gateway
    ├── Audit, outbox, notifications
    └── Integration and reconciliation
        ├── Google Drive adapter
        ├── Calendar adapter
        └── Document import/export workers
```

Use the existing application shell and platform services rather than creating a separate editor application. The inspected project architecture already identifies policy, audit, outbox, files, notifications, search, an AI gateway, and an integration hub as shared components, although their adoption was incomplete in the dated snapshot. [1]

Authority boundaries

Information	Authority
Student draft	Student-owned workspace
Official assignment brief	Institution-authorized course source
Extracted deadline	Unverified candidate until confirmed against its source
Personal study milestone	Student plan
External Drive file	External source unless explicitly imported as a native copy
AI-generated assignment	Draft until an authorized instructor publishes it
AI rubric feedback	Advisory analysis
Official grade	Authorized grading workflow and designated grade ledger
Marketplace enrollment	Commercial access entitlement, not automatically university enrollment

These distinctions must appear in both the data model and interface.

2. Architecture plan for the editor modules

Shared editor shell

All editors should reuse:

- Title, course, assignment, and project context.
- Save/sync indicator.
- Collaborator access.
- Comments and review panel.
- Source browser.
- Version history.
- Export controls.
- Applicable AI policy.
- Keyboard shortcuts and accessibility controls.
- Offline/conflict recovery.

The existing student-screen template provides a reusable shell direction with side rail, context bar, and source-labelled object cards. It does not prove functional editing or persistence. [2]

Editor module boundaries

Module	Owns	Must not own
Docs	Structured text, document layout, citations, suggestions	A separate sharing system or assignment database
Sheets	Cells, formulas, workbook structure, charts	Unbounded execution or private copies of canonical datasets
Slides	Slides, shapes, layouts, notes, presentation structure	Duplicated media storage or independent asset permissions
Research Studio	Questions, screening, evidence extraction, claim-source relationships	Fabricated citations or unrestricted retrieval
Assignment workspace	Stages, deliverable links, reviews, submission preparation	A second document editor
Export service	Format conversion, compatibility findings, output manifests	Authority to publish or submit without permission

Proposed editor adapter contract

```
interface EditorAdapter<T> {
  artifactType: "document" | "workbook" | "presentation";

  validate(content: unknown): ValidationResult;
  normalize(content: T): T;
```

```
  applyOperation(
    content: T,
    operation: EditorOperation
  ): T;

  extractText(content: T): SearchableText;
  collectReferences(content: T): SourceReference[];

  checkAccessibility(content: T): AccessibilityFinding[];

  export(
    version: ArtifactVersion<T>,
    format: ExportFormat
  ): Promise<ExportJob>;
}
```

These are proposed interfaces, not verified repository APIs.

Native content formats

Use versioned schemas rather than treating an imported Office file as the editor's entire internal state.

Editor	Native representation
Docs	Structured document tree with stable block IDs
Sheets	Workbook structure, typed cells, formulas, named ranges, chart definitions
Slides	Slide tree, element IDs, geometry, styles, notes, reading order
Research	Structured project and evidence records linked to artifact versions

Stable block and element IDs allow comments, rubric findings, and citations to remain anchored through edits.

Collaboration and persistence

Recommended design:

1. Apply local operations for responsive editing.
2. Authenticate collaboration sessions.
3. Authorize every incoming operation.
4. Append accepted operations to a durable log.
5. Periodically create immutable snapshots.
6. Expose saved-version receipts.
7. Recover unsynced work after reconnecting.
8. Revoke active editing access when grants are revoked.

For documents, evaluate a CRDT-based collaboration engine. For spreadsheets and slides, use a documented conflict policy appropriate to cells and elements rather than assuming one

collaboration algorithm solves every editing problem.

Do not treat an in-memory collaboration session as durable storage.

Import/export behavior

Each conversion needs:

- Source file identity and version.
- Supported-feature matrix.
- Conversion warnings.
- Output checksum.
- Exported artifact version.
- Accessible-output findings.
- Download authorization.

“Export to Word/Excel/PowerPoint” means the supported content opens correctly in those applications. It does not imply perfect round-trip compatibility.

3. Core data model for assignments and documents

The following is a logical schema to reconcile against existing tables before adding migrations.

Ownership foundation

Use an ownership scope that can represent personal work and institutional work without pretending every private student draft belongs to an institution.

```
ownership_scope
  id
  kind: personal | institutional
  owner_user_id          -- required for personal scope
  institution_id         -- required for institutional scope

workspace
  id
  ownership_scope_id
  kind: personal | course | project | organization
  title
  created_by
  archived_at

workspace_membership
  workspace_id
  principal_id
  role
  granted_by
  revoked_at
```

Database constraints must enforce valid scope combinations. Tenant context and scope authorization must be resolved server-side.

Academic and assignment entities

Entity	Important fields
`academic_term`	Institution, name, start/end dates, timezone
`course_offering`	Course, term, section, institution, authority source
`course_membership`	Offering, user, role, active status
`assignment`	Offering, title, current brief version, publication status, source authority
`assignment_version`	Brief, instructions, rubric version, AI-policy version, publication timestamp
`assignment_deadline`	Assignment, due instant, timezone, audience scope, authoritative source
`learner_assignment`	Assignment, learner/group, working state
`assignment_stage`	Learner assignment, stage, order, estimated effort
`task`	Stage, assignee, planned dates, status, dependencies
`deliverable_requirement`	Assignment version, required type, format, constraints
`deliverable_attachment`	Learner assignment, requirement, artifact
`submission`	Learner assignment, destination, state, receipt
`submission_item`	Submission, exact artifact version, checksum

A student's progress state belongs on `learner_assignment`, not the shared course assignment.

Per-learner deadline overrides should expose the applicable deadline without broadly revealing the reason for an accommodation. The inspected Course Studio template explicitly models individualized deadlines while withholding accommodation reasons, though it remains sample/template evidence.[3]

Document and artifact entities

```
artifact
  id
  workspace_id
  kind
  title
  created_by
  current_version_id
  classification
  lifecycle_state

artifact_version
  id
  artifact_id
  version_number
```

```
  schema_version
  content_blob_ref
  content_hash
  created_by
  created_at
  parent_version_id

artifact_relation
  source_artifact_id
  target_entity_type
  target_entity_id
  relation_type

access_grant
  artifact_id
  principal_type
  principal_id
  permission
  expires_at
  revoked_at

comment
  artifact_id
  anchor
  author_id
  body
  resolved_at
```

`artifact_relation` can connect a document to an assignment, research project, source, dataset, or presentation without duplicating it.

Sources and research

Entity	Purpose
`source_record`	Bibliographic metadata, location, access status, provenance
`source_version`	Exact retrieved/imported version and checksum
`research_project`	Question, protocol, workspace, owner
`screening_decision`	Include/exclude decision and reason
`evidence_excerpt`	Exact quotation or extraction with locator
`research_claim`	Claim text and status
`claim_evidence_link`	Supporting, conflicting, or contextual evidence
`citation_audit`	Missing locators, unmatched quotations, unresolved references

A citation should point to a known source version where possible, not merely a mutable URL.

Essential constraints

- Immutable published assignment versions.
- Immutable submission bundles.
- Same-scope checks or explicit authorized cross-scope sharing.
- Unique external identifiers within each integration connection.
- No deletion of referenced evidence without an explicit retention policy.
- Version-aware comments and rubric reviews.
- No official grade release from an advisory review table.

4. Automated assignment tracking and semester planning

Intake pipeline

```
Authorized course source / selected syllabus / manual input
    ↓
Parse and preserve source
    ↓
Extract candidate assignments and deadlines
    ↓
Deduplicate and detect conflicts
    ↓
Verify or request review
    ↓
Create learner assignment records
    ↓
Suggest milestones and study blocks
    ↓
Student approves plan
    ↓
Action Center and optional calendar publication
```

What automation may do

- Detect assignment mentions in selected materials.
- Propose task breakdowns.
- Track due-date changes.
- Suggest study sessions based on available time.
- Flag missing deliverables.
- Remind students about unresolved reviews.
- Recalculate plans when a verified deadline changes.

What it must not infer

- Opening a document does not mean an assignment is complete.
- Editing activity does not prove learning or contribution.
- Exporting a file does not prove submission.
- A generated schedule is not an institutional deadline.
- A change in a syllabus draft must not silently overwrite a published due date.

Planner model

Add:

```
planning_preference
  user_id
  timezone
  working_hours
  notification_preferences
  accessibility_preferences

study_block
  user_id
  task_id
  planned_start
  planned_end
  status
  origin: manual | suggested | approved

deadline_change
  assignment_id
  previous_deadline_id
  new_deadline_id
  source_version_id
  verification_state
```

Use the original timezone and date semantics for date-only deadlines; do not arbitrarily convert them into midnight UTC.

5. Google Drive synchronization

Three explicit modes

Mode	Behavior
Reference	Link to an external file without creating a native editable copy
Import	Create a native artifact from a selected external version
Managed export	Publish selected Semester versions to a designated external file

Defer generalized bidirectional editing until conversion fidelity and conflict behavior are demonstrated. Do not advertise real-time collaborative editing across Semester and Google

editors without proving it.

Authorization

Prefer per-file access using `drive.file` and a user-selected file workflow. Google documents this scope as granting access to files created by the app or explicitly opened/shared with it, rather than unrestricted Drive access.[4]

Institutional Google Workspace settings may also restrict third-party Drive API access. Treat administrator approval as a potential activation dependency.[5]

Sync records

```
integration_connection
  id
  principal_id
  provider
  credential_vault_ref
  approved_scopes
  state

external_object_link
  connection_id
  external_file_id
  artifact_id
  sync_mode
  last_external_version
  last_native_version
  last_success_at

sync_cursor
  connection_id
  source_scope
  cursor
  updated_at

sync_job
  id
  external_object_link_id
  direction
  state
  idempotency_key
  error_code

sync_conflict
  external_object_link_id
  base_version
  native_version
  external_version
  resolution
```

Change handling

Google Drive supports push notifications for `files` and `changes`, with HTTPS receivers and notification channels. Its changes interface can include removed files and access-loss events.[6] [7]

Proposed processing:

1. Validate the notification against the registered connection/channel.
2. Queue reconciliation.
3. Retrieve the relevant change information.
4. Check access and sync scope.
5. Compare the last synchronized versions.
6. Import, export, or create a conflict.
7. Update the cursor only after durable processing.
8. Periodically reconcile to recover missed events.

Google's private `appProperties` can store an app-specific object identifier. Use them as mapping hints—not as authorization or the sole synchronization ledger.[8]

External writes, including enabling export to an existing Drive file, require approval of the resolved target and exact intended change.

6. Course marketplace and Research Studio

Course marketplace model

Treat "course marketplace" as discovery and access to approved educational offerings, not a substitute for official university registration.

```
course_listing
  publisher_id
  course_template_id
  title
  description
  audience
  moderation_state
  publication_state

listing_version
  listing_id
  content_manifest
  rights_evidence
  accessibility_review

offer
  listing_id
  access_terms
  price
```

```
  currency

learning_entitlement
  user_id
  listing_version_id
  access_start
  access_end
  grant_source
```

Keep commercial purchase, learning entitlement, institution enrollment, credit award, and credential issuance separate.

Assignment generator

Support distinct modes:

- Instructor authoring: generate a draft brief, rubric, examples, and instructor-only marking material.
- Student practice: generate ungraded exercises and practice papers.
- Student planning: generate stages and checklists from an existing official assignment.

Publishing requires instructor authorization. A student practice generator must not reveal instructor-only keys or reinterpret practice content as an official course assignment.

Rubric scoring

Use three different records and permissions:

Mode	Output	Authority
Student self-check	Evidence checklist and gaps	Student advisory
AI-assisted scoring	Proposed criterion scores and rationale	Reviewer recommendation
Official assessment	Approved scores and released feedback	Authorized grader

Recommended assessment fields:

```
rubric_review
  submission_id
  rubric_version_id
  review_mode
  reviewer_id
  model_run_id
  state

criterion_assessment
  review_id
  criterion_id
  evidence_anchors
  proposed_score
  approved_score
```

```
  rationale
  approved_by
```

Student self-check records should not contain grade predictions. AI-assisted scoring should remain unreleased until an authorized human approves it.

The Course Studio template depicts rubrics, assignment AI restrictions, accessibility gates, and controlled grade release, but its release handler only changes local component state. It is a design reference, not evidence of a production grading workflow. [3]

7. How the policy-controlled AI layer works

Policy inputs

Resolve:

- Authenticated user and acting role.
- Personal or institutional ownership scope.
- Course and assignment.
- Requested operation.
- Data classification.
- Consent and sharing grants.
- Institution policy.
- Course policy.
- Assignment policy.
- User preferences.
- Approved provider/model.
- Cost and execution limits.

A lower-level policy may narrow permission but must not silently expand a restriction imposed above it.

Request flow

```
AI request
    ↓
Authenticate and resolve context
    ↓
Authorize access to objects
    ↓
Compute effective policy
    ↓
Retrieve only permitted source versions
    ↓
Minimize/redact provider payload
    ↓
```

```
Authorize provider, operation, and budget
    ↓
Execute bounded model request
    ↓
Validate citations and output
    ↓
Return advisory result
    ↓
Approve any proposed write separately
    ↓
Recheck policy at execution
    ↓
Execute permitted action and audit receipt
```

Proposed decision contract

```
type AIPolicyDecision = {
  decisionId: string;
  allowed: boolean;
  policyVersionIds: string[];

  permittedOperations: Array<
    "explain" |
    "summarize" |
    "draft_feedback" |
    "practice_generate" |
    "instructor_author" |
    "score_recommendation"
  >;

  sourceVersionIds: string[];
  providerAllowlist: string[];
  prohibitedDataClasses: string[];

  requiresHumanReview: boolean;
  requiresActionConfirmation: boolean;
  expiresAt: string;
  reasonCodes: string[];
};
```

The model cannot create or modify its own permission decision.

Operation-specific rules

Operation	Rule
Concept explanation	Allowed only under applicable course/assignment rules
Draft feedback	Review student-authored material within the allowed scope
Graded-answer generation	Deny where assignment policy forbids it
Practice generation	Keep distinct from official assessments and protected keys

Operation	Rule
Instructor assignment drafting	Instructor capability required
Rubric scoring recommendation	Approved workflow and human review required
Drive export	Resolve target, show content, obtain confirmation
Submission	Separate authorization and verified receipt
Official grade release	Human authority and institution-approved command path

Enforcement location

Policy checks must occur:

- Before source retrieval.
- Before provider egress.
- Before every agent tool call.
- Before queued jobs execute.
- Before a write is committed.
- After policy changes invalidate existing sessions or decisions.

The dated project snapshot reports that the AI edge function checked a global kill switch but did not enforce the effective tenant AI policy. Repairing and verifying that call-boundary enforcement is a prerequisite—not an optional interface improvement.[1]

Prompt-injection and privacy controls

- Treat documents, marketplace content, and retrieved sources as untrusted data.
- Do not let instructions inside a file override system policy.
- Never expose raw credentials to a model.
- Constrain tools to validated operations and resolved object IDs.
- Filter retrieval by permissions before ranking.
- Recheck permissions before returning excerpts.
- Store minimal audit metadata rather than private prompts by default.
- Keep provider restrictions explicit; no unapproved fallback provider.
- Disable external provider calls when data cannot be safely minimized under policy.

8. Implementation gates and first acceptance scenario

Workstream	Proposed owner	Dependency	Required evidence
Ownership and artifact persistence	Platform lead	Existing schema reconciliation	Personal/tenant isolation, grant revocation, restore
Editors and export	Workspace lead	Artifact version service	Editing, recovery, conversion fixtures

Workstream	Proposed owner	Dependency	Required evidence
Assignment intake and planner	Learning lead	Source/version model	Verified deadlines, conflict handling, student approval
Drive sync	Integration lead	OAuth, vault, reconciliation	Scope limits, access-loss handling, retry/conflict tests
AI gateway	Security/AI lead	Policy engine and data classification	No forbidden egress or tool execution
Rubric workflows	Assessment lead	Immutable submission/rubric versions	Human approval and controlled release
Marketplace	Trust/product lead	Rights, moderation, entitlement model	Approved publication and access lifecycle

First end-to-end acceptance test

1. Student selects a syllabus in Drive.
2. Semester imports an authorized version.
3. Intake proposes assignments with source-linked deadlines.
4. Student verifies ambiguous entries.
5. Planner creates editable milestones.
6. Student drafts in Docs and analyzes data in Sheets.
7. Research Studio links claims to selected evidence.
8. AI provides only the assistance permitted by that assignment.
9. Rubric self-check identifies gaps without predicting a grade.
10. Semester exports the exact reviewed version to a student-approved target.
11. A separate authorized submission workflow produces a verified receipt.

Test failures as seriously as success: revoked Drive access, conflicting deadlines, offline edits, unsupported workbook formulas, changed AI policy, rejected export, and unavailable submission destination.

This architecture delivers the requested native editors, assignment/document model, planner, course ecosystem, Drive bridge, and governed AI layer without confusing drafts with official records or interface designs with completed implementation.

⁂

1. The Main Semester design system (2) copy/uploads/Semester build Architecture.pdf
2. The Main Semester design system (2) copy/templates/student-screen/StudentScreen.dc.html
3. The Main Semester design system (2) copy/templates/course-studio/CourseStudio.dc.html
4. https://developers.google.com/workspace/drive/api/guides/api-specific-auth
5. https://knowledge.workspace.google.com/admin/drive/allow-third-party-apps-for-drive-files
6. https://developers.google.com/workspace/drive/api/guides/push

7. https://developers.google.com/workspace/drive/api/reference/rest/v3/changes/watch
8. https://developers.google.com/workspace/drive/api/guides/properties
9. https://workspace.google.com/marketplace/app/drive_watcher/486176815363
10. https://workspaceupdates.googleblog.com/2025/07/google-drive-events-api-now-available.html
11. https://googleapis.dev/dotnet/Google.Apis.Drive.v3/latest/api/Google.Apis.Drive.v3.Data.File.html
12. https://workspaceupdates.googleblog.com/search/label/Rapid Release
13. https://workspaceupdates.googleblog.com/2026/
14. https://stackoverflow.com/questions/22403014/google-drive-api-scope-and-file-access-drive-vs-drive-files
15. https://issuetracker.google.com/issues/305065929
16. https://stackoverflow.com/questions/18144094/google-drive-sdk-push-notifications-watch-changes-on-all-files
17. https://stackoverflow.com/questions/66458874/google-drive-api-file-watch-does-not-notify
18. https://github.com/transloadit/uppy/issues/4793
19. The Main Semester design system (2) copy/uploads/semester_gtm_playbook.pdf